

Nugis

Documentazione tecnica

Installer · Eseguibile · Licenze · Bot Telegram

Generato il 29/08/2026

Installer Windows (NugisSetup.exe)

Non va confuso con `Nugis.exe` (vedi `docs/exe-packaging.md`): sono due cose diverse.

```
NugisSetup.exe  <- lo lanci UNA VOLTA sul PC del cliente. Copia i file, chiede le
                  credenziali admin e la licenza, poi non serve più.
Nugis.exe       <- l'app vera, quella che l'installer mette dentro
                  Program Files\Nugis e fa partire (bot + API + pannello web,
                  tutto in un solo processo).
```

Come si costruisce

```
npm run build:installer -w apps/server
```

Fa due cose in sequenza (`scripts/build-installer.js`): 1. Builda `Nugis.exe` (`scripts/build-exe.js`, stesso processo di `docs/exe-packaging.md`) 2. Lo pacchettizza con **Inno Setup 6** (`installer/nugis.iss`) in un unico `installer-output/NugisSetup.exe`

Richiede Inno Setup 6 installato (`ISCC.exe`) sulla macchina dove fai la build — non sulla macchina del cliente, solo su quella dello sviluppatore/ufficio.

Per una build di test più veloce (salta l'offuscamento):

```
SKIP_OBFUSCATE=1 npm run build:installer -w apps/server
```

Cosa fa l'installer, in ordine

1. Chiede la cartella di installazione (default `Program Files\Nugis`)
2. **Pagina "Account amministratore"**: email + password (min. 8 caratteri) + conferma. Questa email **non è usata per inviare nulla** — nel progetto non esiste nessuna configurazione SMTP/invio email (verificato: zero occorrenze di `nodemailer/smtp` nel codice). È solo lo username di login per il pannello web, salvato nel database locale di quell'installazione.
3. **Pagina "Licenza" (opzionale)**: indirizzo del license-server dell'ufficio + chiave `NUGIS-XXXX-...`. Si può lasciare vuota per un uso interno senza restrizioni — vedi `docs/license-server.md` per come generarne una.
4. Copia i file, poi **genera da solo il .env**: `DATABASE_URL` assoluto puntato dentro la cartella di installazione, `ADMIN_JWT_SECRET` ed `ENCRYPTION_KEY` casuali (generati con l' RNG crittografico di PowerShell, non un generatore debole) — questi due non vengono mai chiesti all'utente, sono segreti tecnici, non credenziali da scegliere o ricordare.
5. Registra un'**attività pianificata di Windows** (`schtasks`) che avvia `Nugis.exe` all'accensione del PC, come account SYSTEM, in sessione non interattiva: niente finestra console visibile, e se il processo termina riparte da solo (fino a 3 tentativi/minuto).
6. Avvia subito l'attività e apre il browser su `http://localhost:4000`.

La disinstallazione (dal Pannello di controllo o dal collegamento nel menu Start) ferma il processo, rimuove l'attività pianificata e cancella i file installati.

Perché un'attività pianificata e non un vero "servizio Windows"

Un vero servizio Windows richiede che l'eseguibile parli il protocollo SCM (Service Control Manager) di Windows — il nostro non lo fa, essendo un normale processo Node impacchettato con pkg. La via più comune per aggirarlo è un wrapper di terze parti (es. NSSM), ma imbarcare un binario non firmato di terze parti in un installer commerciale è un compromesso di fiducia/supply-chain che non ho voluto prendere senza un confronto esplicito con chi vende il prodotto. L'attività pianificata con `/RU SYSTEM /SC ONSTART` ottiene lo stesso risultato pratico (avvio automatico prima del login, nessuna finestra, riavvio dopo crash) senza dipendenze esterne.

Stato del collaudo (onesto)

- **Compilazione:** verificata — Inno Setup compila senza errori includendo tutti i file attesi (se un sorgente mancasse, la compilazione fallirebbe). Durante lo sviluppo dello script sono stati trovati e corretti due bug reali di sintassi Pascal (`GetTickCount` non esiste in Inno Setup Pascal Script; uso scorretto del valore di ritorno di `StringChangeEx`, che modifica la stringa per riferimento e ritorna un Integer, non una String).
- **Esecuzione reale del wizard NON testata:** l'installazione vera (elevazione UAC, scrittura in Program Files, generazione `.env`, registrazione dell'attività pianificata, disinstallazione) richiede di modificare il sistema della macchina di sviluppo (installare in Program Files, creare un task SYSTEM) — è stato deciso di **non farlo automaticamente** senza conferma esplicita, e al momento della stesura di questo documento quella conferma non è ancora arrivata. **Prima di consegnare l'installer a un cliente vero, va fatto almeno un giro di test completo su una macchina Windows pulita** (o una VM), verificando in particolare: il wizard scrive un `.env` valido, `Nugis.exe` parte davvero dall'attività pianificata dopo un riavvio del PC, e la disinstallazione non lascia residui.

Cosa manca ancora per un prodotto commercialmente "pulito"

- **Firma digitale (code-signing):** né `Nugis.exe` né `NugisSetup.exe` sono firmati. Windows SmartScreen mostrerà un avviso "Editore sconosciuto" al cliente. Serve un certificato Authenticode (tipicamente 70-300€/anno) per rimuoverlo.
- **Icona personalizzata** per l'installer e l'eseguibile (oggi usano quella di default di Inno Setup/pkg).
- Vedi anche `docs/license-server.md`, sezione finale, per cosa manca lato gestione licenze (nessuna UI, solo API dirette).

Distribuzione come eseguibile Windows (.exe)

Come funziona

`npm run build:exe -w apps/server` produce una cartella `apps/server/dist-exe/` con:

<code>dist-exe/</code>	
<code>Nugis.exe</code>	<- avvialo, fa partire API + pannello web insieme
<code>public/</code>	<- build statica del pannello (servita dallo stesso <code>.exe</code>)
<code>generated/prisma-client/</code>	<- client Prisma + motore nativo
<code>prisma/migrations/</code>	<- applicate automaticamente all'avvio
<code>.env.example</code>	<- copia in <code>.env</code> e compila prima del primo avvio

Non è un singolo file autosufficiente al 100% — Prisma richiede un motore nativo (query engine, un file binario) che `pkg` non può fondere dentro l'eseguibile. È normale per applicazioni Node.js pacchettizzate: si distribuisce una cartella, non un unico file. Il pannello web invece è servito direttamente dallo stesso processo (niente nginx/secondo eseguibile separato, a differenza della versione Docker).

Perché non è DRM vero (va detto chiaramente al cliente/socio, non solo letto qui)

Il codice viene offuscato (`javascript-obfuscator`) prima di essere impacchettato con `pkg`: non è più testo leggibile in chiaro come il progetto sorgente, e non è estraibile con un semplice editor di testo. Ma: - Esistono strumenti pubblici per "spacchettare" i binari `pkg` e risalire al codice offuscato - Il codice offuscato, anche se difficile da leggere, resta deoffuscabile con impegno e strumenti dedicati

La protezione reale resta il contratto di licenza commerciale, non questo meccanismo tecnico. Il controllo licenza (`apps/license-server`) serve a far scadere/revocare un cliente che smette di pagare tramite un server centrale che *voi* controllate — non a rendere il software "inviolabile".

Requisiti per fare la build

- Windows con Node.js 20+ (la build va fatta su Windows per un output `win-x64`)
- Connessione internet la prima volta (`pkg` scarica un binario Node precompilato, poi lo mette in cache locale — build successive sono più veloci)
- **Nessun compilatore C++/Visual Studio necessario SE il binario precompilato è disponibile** per la combinazione versione Node + piattaforma scelta. Se `pkg` non lo trova nella sua cache remota, prova a compilare Node da sorgente — richiesto Visual Studio Build Tools con i workload C++, un processo lungo (30+ minuti) e pesante. Se capita, conviene cambiare la versione target Node (es. `node20` invece di `node18`) prima di installare un intero toolchain di compilazione.

Build più veloci in fase di test

```
SKIP_OBFUSCATE=1 npm run build:exe -w apps/server
```

Salta l'offuscamento (che rallenta la build e non serve mentre stai ancora testando che tutto funzioni) — usalo solo per le build che consegnerai davvero al cliente.

Prima del primo avvio sulla macchina del cliente

Se distribuisce solo la cartella `dist-exe/` senza installer (vedi `docs/installer.md` per il caso normale, con installer):

1. Copia `.env.example` in `.env` dentro `dist-exe/`
2. Compila almeno: `ADMIN_JWT_SECRET`, `ADMIN_DEFAULT_EMAIL`, `ADMIN_DEFAULT_PASSWORD`, `ENCRYPTION_KEY` — occhio a non lasciare righe duplicate della stessa chiave nel file: `dotenv` usa la **prima** occorrenza, quindi se `.env.example` ha già un placeholder per una chiave e ne aggiungi un'altra riga sotto con `>>`, vince quella sbagliata in cima. Meglio rigenerare il file da zero.
3. `DATABASE_URL` deve essere un **path assoluto** (es. `file:C:/Nugis/nugis.db`), non relativo — dentro lo snapshot di pkg un path relativo non si risolve in modo affidabile.
4. Se è un'installazione venduta con licenza: `LICENSE_SERVER_URL` e `LICENSE_KEY` (vedi `docs/license-server.md`)
5. Avvia `Nugis.exe`

Cosa fa da solo all'avvio (testato dal vivo, non solo scritto)

Nessun passaggio manuale di `prisma migrate` o di seed: `Nugis.exe`, alla partenza:

1. **Applica le migrazioni Prisma pendenti** (`services/autoMigrate.ts`) leggendo direttamente i file `migration.sql`, perché dentro l'exe non esiste una CLI Prisma installata. Verificato con tutte e 13 le migrazioni presenti al 29/08/2026.
2. **Crea il primo admin** (`services/bootstrapAdmin.ts`) da `ADMIN_DEFAULT_EMAIL/ADMIN_DEFAULT_PASSWORD` — solo se non esiste già nessun admin nel DB (così non resuscita credenziali di default dopo che il cliente ne ha create di sue).
3. Controlla la licenza, poi fa partire i bot Telegram delle creator configurate.

Test end-to-end confermato: `/health` → tutte le migrazioni applicate → admin creato → login → pannello web servito dallo stesso processo su `http://localhost:4000`.

Server di Licenze (per l'ufficio che vende Nugis)

Strumento interno, separato dal prodotto Nugis vero e proprio: lo gestisce solo chi vende le licenze (l'ufficio), i clienti finali non ci accedono mai direttamente.

Testato dal vivo (creazione → validazione → revoca → eliminazione, ciclo completo con richieste HTTP reali), non solo scritto e compilato.

Come funziona, in breve

```
[Ufficio] server di licenze (apps/license-server) – genera chiavi, le gestisce
        ↑ POST /validate (ogni ~6h, automatico)
[Cliente] installazione Nugis (apps/server) – verifica di essere ancora valida
```

Se un'installazione Nugis **non** ha `LICENSE_SERVER_URL/LICENSE_KEY` configurati nel suo `.env`, il controllo è semplicemente disattivato — nessuna restrizione. Il controllo scatta SOLO se configurato: è pensato per le installazioni vendute, non per l'uso interno.

Setup del server di licenze (una volta sola, lo ospita l'ufficio)

```
cd apps/license-server
cp .env.example .env
# apri .env e imposta LICENSE_ADMIN_SECRET (stringa lunga e casuale, es.
# node -e "console.log(require('crypto').randomBytes(24).toString('hex'))")

npm run prisma:generate -w apps/license-server
npx prisma migrate deploy --schema apps/license-server/prisma/schema.prisma
npm run dev:license-server
```

Bug corretto (29/08/2026): `DATABASE_URL` in `.env.example` puntava a file: `./prisma/licenses.db`. Prisma risolve i path relativi di `DATABASE_URL` rispetto alla cartella dove sta `schema.prisma` (cioè `prisma/`), non rispetto alla working directory da cui lanci il comando — quel valore creava quindi il file in `prisma/prisma/licenses.db` invece che in `prisma/licenses.db`. Il valore corretto, già in `.env.example`, è file: `./licenses.db`.

Va tenuto **sempre acceso e raggiungibile** dalle installazioni dei clienti (un server con IP fisso o un dominio, non un laptop che si spegne la sera) — se è irraggiungibile per più di 5 giorni consecutivi, le installazioni clienti smettono di funzionare (grace period, vedi `services/licensing.ts` in `apps/server`).

Creare una licenza per un nuovo cliente

```
curl -X POST http://IP_UFFICIO:5000/admin/licenses \
-H "Content-Type: application/json" \
-H "x-admin-secret: IL_TUO_LICENSE_ADMIN_SECRET" \
-d '{
  "customerName": "Nome Cliente SRL",
  "expiresAt": "2027-01-01T00:00:00Z",
  "maxCreators": 5,
  "notes": "Piano annuale, rinnovo gennaio"
}'
```

La risposta contiene `"key": "NUGIS-XXXX-XXXX-XXXX-XXXX"` — **copiala subito**, non viene più mostrata per intero dopo (le liste successive la mostrano mascherata). Comunicala al cliente insieme a `LICENSE_SERVER_URL` (l'indirizzo del vostro server) da mettere nel `.env` della sua installazione Nugis.

Gestione licenze esistenti

```
# Elenco (chiavi mascherate)
curl http://IP_UFFICIO:5000/admin/licenses -H "x-admin-secret: ..."

# Estendere la scadenza / cambiare il limite creator
curl -X PATCH http://IP_UFFICIO:5000/admin/licenses/ID \
-H "x-admin-secret: ..." -H "Content-Type: application/json" \
-d '{"expiresAt": "2028-01-01T00:00:00Z", "maxCreators": 10}'

# Revocare (il cliente smette di funzionare al prossimo check, entro 6h)
curl -X PATCH http://IP_UFFICIO:5000/admin/licenses/ID \
-H "x-admin-secret: ..." -H "Content-Type: application/json" \
-d '{"status": "revoked"}'
```

Ogni licenza tiene traccia di `lastCheckAt/lastCheckInstanceId`: utile per vedere se e quando un cliente ha effettivamente controllato l'ultima volta.

Lato installazione cliente (Nugis)

Nel `.env` di `apps/server` del cliente:

```
LICENSE_SERVER_URL="http://IP_UFFICIO:5000"
LICENSE_KEY="NUGIS-XXXX-XXXX-XXXX-XXXX"
```

Da quel momento: verifica al boot + ogni 6 ore. Se non valida, i bot Telegram non partono e il pannello mostra un banner rosso in cima a ogni pagina — i dati del cliente NON vengono toccati/cancellati, solo il servizio si ferma.

Limiti onesti di questo sistema

- **Non è DRM vero**: chi ha accesso al codice sorgente di Nugis può, tecnicamente, rimuovere il controllo licenza. La protezione reale è il contratto commerciale, non la tecnologia — vedi la discussione su come "chiudere in un applicativo" per il quadro completo.
- **Nessuna interfaccia grafica** per gestire le licenze: solo curl/API dirette. Se servono più licenze da gestire regolarmente, vale la pena costruire una piccola UI sopra queste stesse route (sono già pronte).
- **Un solo segreto condiviso** per l'accesso admin (`LICENSE_ADMIN_SECRET`), non un vero sistema di utenti multipli — adeguato per un piccolo ufficio, non per un team grande con necessità di permessi differenziati.

Collegare il bot Telegram di ogni creator

Ogni creator ha il **proprio** bot Telegram — non è una configurazione unica per tutta l'installazione Nugis, va ripetuta per ciascuna.

Flusso normale (per ogni nuova creator)

1. Crea la creator nel pannello web di Nugis
2. Su Telegram, apri una chat con **@BotFather**, crea un nuovo bot (`/newbot`), copia il **token** che ti dà (formato `123456789:ABC-DEF1234...`)
3. Incolla quel token nella scheda della creator, nel pannello web (campo token bot) — **non** va messo nel file `.env`
4. Nugis lo salva **cifrato** nel database (`encryptCredential`, vedi `apps/server/src/routes/creators.ts`) e fa partire il bot di quella creator in automatico

La riga `TELEGRAM_BOT_TOKEN` dentro `.env` è solo un **fallback opzionale** per un eventuale bot "gateway" unico condiviso — il percorso normale/atteso è un token per creator, inserito dal pannello.

Cosa è diverso: le "cartelle chat reali" (MTProto)

Il brief chiede anche di poter aggiungere automaticamente i fan a una vera cartella chat di Telegram (funzione avanzata, opzionale). Questa parte:

- **Non usa il token del bot**, ma un vero **account Telegram personale** dell'operatore (numero di telefono + login one-time con SMS + eventuale password 2FA)
- Richiede `TELEGRAM_API_ID/TELEGRAM_API_HASH` ottenuti gratis su <https://my.telegram.org/apps>, configurati **una sola volta** nel `.env` dell'installazione (condivisi, non per-creator)
- Il login vero e proprio (numero + codice SMS) si fa dal pannello, sezione "Fan & Cartelle → Cartelle Telegram Reali" — una tantum, non automatizzabile (il codice arriva sul telefono dell'operatore)

Stato di collaudo: il codice è scritto e segue la documentazione ufficiale della libreria (`telegram/GramJS`), ma **non è mai stato testato con un account Telegram reale** — richiede un vero numero di telefono e un vero SMS ricevuto, cosa che non è simulabile in un ambiente di sviluppo/test automatico. È la parte del progetto con il collaudo più debole: va provata dal vivo con un account reale (anche uno secondario) prima di fare affidamento su di essa in produzione. Vedi anche `docs/telegram-automation-limits.md` per i limiti di invio/spam che si applicano comunque a tutto ciò che manda messaggi via Telegram (bot o account MTProto).